Introduction

Objectif

Nous allons créer le site web d'un petit réseau social. Dans ce module, nous nous concentrerons sur la génération de code côté serveur, à l'aide du langage PHP.

Une démonstration est disponible à l'adresse https://epfc.verhaegen.me/msn/.

Prenez le temps de "jouer" avec l'application pour bien en comprendre les fonctionnalités car vous allez devoir les reproduire.

Prérequis

Vous devez avoir installé l'environnement de travail conformément au tutoriel correspondant (voir Moodle).

Commencez par télécharger le fichier `my_social_network_base.zip`, décompressez-le, copiez le dossier dans le sous-dossier `projects` de votre environnement et ouvrez ce projet dans Visual Studio Code.

À l'aide de phpMyAdmin, importez le fichier `my_social_network_base.sql` qui contient le script de création d'une base de données (`my_social_network_base`) avec une seule table `Members`. Cette table a le format et le contenu suivant :

pseudo (clé primaire)	password	profile	picture_path
admin	admin	Je suis l'administrateur	admin.png
ben	ben	Je suis Benoît.	ben.png

Constatez enfin que tout fonctionne bien en accédant à la ressource : http://localhost/my_social_network_base/test.php

Important

N'oubliez pas de copier ce projet dans votre répertoire personnel en fin de cours : nous y travaillerons pendant quelques semaines.

Liste des membres

Nous allons créer un document PHP qui permet d'afficher la liste des membres de notre réseau social.

Nous vous recommandons d'éviter de copier/coller le code proposé dans ce tutoriel, mais plutôt de prendre le temps de le retaper : cela vous offrira l'opportunité de bien analyser et de bien comprendre chaque ligne.

Ouvrez le fichier `members.php` dans votre projet VSCode et ajoutez-y le contenu suivant au-dessus du document, avant la toute première balise :

```
<?php
    try
    {
        $pdo = new
PDO("mysql:host=localhost;dbname=my_social_network_base;charset=utf8mb4",
"root", "root");
        $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);


        $query = $pdo->prepare("SELECT pseudo FROM Members");
        $query->execute();
        $members = $query->fetchAll();
    }
    catch (Exception $exc)
    {
        die('Error while accessing database. Please contact your
administrator.');
    }
?>
```

Ce morceau de code PHP se connecte à la base de données, l'interroge et produit un tableau contenant les pseudos de la table `Members`.

Analysons chaque partie de code :

Connexion à la base de données

```
$pdo = new
PDO("mysql:host=localhost;dbname=my_social_network_base;charset=utf8mb4",
"root", "root");
```

Cette ligne crée un *PDO* (un *PHP Data Object*), un objet permettant d'interagir avec la base de données.

> Il existe d'autres librairies comme *MySQLi* mais *PDO* permet de s'abstraire du type de base de données (MySQL, Oracle, Microsoft SQLServer, ...).

Le constructeur de la classe `PDO` prend en paramètre le nom de la source de données (*Data Source Name* : type de base de données, serveur, nom de la base de données et encodage) ainsi qu'un nom d'utilisateur et un mot de passe. Une connexion à la base de données est établie lors de la construction de cet objet.

Configuration de la connexion PDO

```
$pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
```

Cette ligne change un attribut de la connexion PDO afin que les requêtes erronées déclenchent des exceptions. Il sera ainsi plus facile de gérer ces éventuelles erreurs.

Gestion des erreurs de connexion

```
<?php
    try
    {
        $pdo = new
PDO("mysql:host=localhost;dbname=my_social_network_base;charset=utf8mb4",
"root", "root");
        $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);

        $query = $pdo->prepare("SELECT pseudo FROM Members");
        $query->execute();
        $members = $query->fetchAll();
    }
    catch (Exception $exc)
    {
        die('Error while accessing database. Please contact your
administrator.');
    }
?>
```

Les instructions qui manipulent la base de données sont englobées dans un bloc `try` : en cas d'erreur, une *exception* (plus de détails dans le cours PRO2) est déclenchée, le code du bloc `try` est interrompu et le code du bloc `catch` est exécuté. Dans notre cas, l'entièreté du script est interrompue (`die()`) et un message est produit.

> L'utilisation de `die()` n'est pas une bonne pratique, mais nous l'utiliserons dans un premier temps.

Exécution d'une requête

```
$query = $pdo->prepare("SELECT pseudo FROM Members");
$query->execute();
$members = $query->fetchAll();
```

La méthode (*fonction*) `prepare()` appelée sur l'objet `$pdo` prend en paramètre une requête SQL (*string*), la compile et renvoie un objet "requête" (un `PDOStatement`). La méthode `execute()` exécute la requête, comme son nom l'indique. En cas d'erreur une exception est déclenchée et le bloc `catch` s'exécute.

Pour parcourir plus facilement ce résultat, on peut le transformer en un tableau à deux dimensions (lignes et colonnes) à l'aide de sa méthode `fetchAll()`.

Si tout se passe comme prévu, nous avons donc un tableau à deux dimensions référencé par la variable `$members`.

Si vous affichez ce tableau en ajoutant une ligne `var_dump($members)` (méthode similaire à `print_r`), vous devriez voir le résultat suivant s'afficher en haut de la page :

```
Array
(
    [0] => Array
        (
            [pseudo] => admin
            [0] => admin
        )

    [1] => Array
        (
            [pseudo] => ben
            [0] => ben
        )

)
```

Pour le vérifier, collez l'URL suivante dans votre navigateur : http://localhost/my_social_network_base/members.php.

Nous allons maintenant parcourir ce tableau afin de remplir la liste des membres. Pour ce faire, ajoutez les lignes suivantes dans le code HTML entre les balises `<ul>` et `</ul>` :

```
<?php
    foreach($members as $member){
        echo "<li>".$member['pseudo']."</li>";
   }
?>
```

Ce morceau de code parcourt le tableau `$members` ligne par ligne à l'aide d'une boucle `foreach`. Chaque ligne est un tableau associatif que nous avons assigné à la variable

`$member`. Ce tableau contient une entrée associée à la clé `pseudo`. Pour chaque ligne, nous affichons un élément de liste (`<li></li>`) contenant le pseudo de l'utilisateur en question.

Votre page devrait maintenant afficher une liste contenant deux noms : admin et ben.

Profil d'un utilisateur

Nous allons maintenant créer une page permettant d'afficher le profil d'un utilisateur : son nom, son descriptif et son image.

Le fichier en question est `profile.php` et cette ressource s'attend à recevoir un paramètre `pseudo` via la méthode *GET*. Pour accéder au profil de l'utilisateur *ben*, il faudra accéder à la ressource http://localhost/my_social_network_base/profile.php?pseudo=ben.

Dans un premier temps, il faut vérifier que l'on a bien reçu le paramètre `pseudo` via la méthode *GET*. Si c'est le cas, on retient cette information dans la variable `$pseudo`. Dans le cas contraire, on interrompt le script et on affiche un message d'erreur.

```
if(isset($_GET["pseudo"])){
    $pseudo = $_GET["pseudo"];
}
else{
    die("This page expects a `pseudo` parameter via the GET method");
}
```

Ensuite, on se connecte à la base de données et on récupère l'entièreté de la ligne correspondant au pseudo reçu dans la table `Members`. Analysez les appels de méthodes `prepare`, `execute` et `fetch` et soyez certains de tout comprendre.

```
try
{
    $pdo = new
PDO("mysql:host=localhost;dbname=my_social_network_base;charset=utf8mb4",
"root", "root");
    $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
    $query = $pdo->prepare("SELECT * FROM Members WHERE pseudo = :pseudo");
    $query->execute(["pseudo" => $pseudo]);
    $profile = $query->fetch(); // un seul résultat au maximum
}
catch (Exception $exc)
{
    die("Error while accessing database. Please contact your administrator.");
}
```

Il faut maintenant vérifier que l'utilisateur en question existe bel et bien : soit la base de données a renvoyé un résultat et donc l'utilisateur existe, soit la base de données n'a renvoyé aucun résultat. Dans ce cas, on affiche un message d'erreur. Si l'utilisateur existe, on retient sa description et le chemin vers son image dans deux variables.

```php
if($query->rowCount()==0){
    die("Can't find user '$pseudo' in the database.");
}
else{
    $description = $profile["profile"];
    $picture_path = $profile["picture_path"];
}
```

Finalement, il faut remplir le corps de notre page HTML avec les informations récupérées : le pseudo, la description et l'image. Pour ce faire, modifiez votre document afin d'afficher aux bons endroits ces données.

Raccourci `<?= … ?>`

Pour injecter une valeur PHP dans du HTML, on pourrait écrire `<?php echo $pseudo; ?>`.
PHP propose une forme équivalente plus courte : `<?= $pseudo ?>`.

Les deux font exactement la même chose (afficher la valeur). La forme courte est plus lisible dans les gabarits HTML ; nous l'utiliserons dans la suite du parcours. Elle est activée par défaut depuis PHP 5.4.

```php
<!DOCTYPE html>
<html>
    <head>
        <title><?= $pseudo ?>'s Profile!</title>
        <meta charset="UTF-8">
        <meta name="viewport" content="width=device-width, initial-scale=1.0">
        <link href="styles.css" rel="stylesheet" type="text/css"/>
    </head>
    <body>
        <div class="title"><?= $pseudo ?>'s Profile!</div>
        <div class="menu">
            <a href="profile.php?pseudo=<?= $pseudo ?>">Home</a>
            <a href="members.php">Members</a>
            <a href="friends.php">Friends</a>
            <a href="messages.php">Messages</a>
            <a href="edit_profile.php">Edit Profile</a>
            <a href="logout.php">Log Out</a>
        </div>
        <div class="main">
            <div>
                <?php
                if(mb_strlen($description ?? '') == 0){
                    echo 'No profile string entered yet!';
                } else {
                    echo $description;
                }
                ?>
            </div>
            <div>
                <?php
```

```
                if(mb_strlen($picture_path ?? '') == 0){
                    echo 'No picture loaded yet!';
                } else {
                    echo "<img src=\"$picture_path\" width=\"100\"
alt=\"$pseudo&amp;apos;s picture\">";
                }
                ?>
            </div>
        </div>
    </body>
</html>
```

Enfin, constatez que tout fonctionne bien en suivant les liens suivants :

- http://localhost/my_social_network_base/profile.php?pseudo=ben
- http://localhost/my_social_network_base/profile.php?pseudo=admin

Refactorisation

Vous l'avez remarqué : le code de connexion à la base de données est identique dans les deux documents PHP. Ce "copier-coller" est une très mauvaise pratique car il augmente le risque d'erreur et réduit la lisibilité du code.

Une façon de faire est de l'extraire dans un autre fichier.

Créez le fichier `functions.php` et ajoutez-y les lignes suivantes :

```php
<?php
try
{
    $pdo = new
PDO("mysql:host=localhost;dbname=my_social_network_base;charset=utf8mb4",
"root", "root");
    $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
}
catch (Exception $exc)
{
    die("Error while accessing database. Please contact your administrator.");
}
```

Ces lignes créent une connexion à la base de données et la retiennent dans la variable `$pdo`.

Nous avons donc extrait le code de connexion. Nous allons maintenant importer ce fichier dans les deux documents PHP `members.php` et `profile.php`.

Pour ce faire, modifiez le début de ces **deux** fichiers de la façon suivante :

```php
<?php
require_once "functions.php";

try
{
    //le code de connexion a été supprimé et inséré dans functions.php

    $query = $pdo->prepare("SELECT...
```

Remarquez que nous avons importé le fichier `functions.php` et que nous avons supprimé les deux premières lignes du bloc `try` (les deux lignes qui étaient répétées dans les deux fichiers).

Apportons encore une modification à notre fichier `functions.php` : afin de le rendre plus lisible et de pouvoir changer les informations de connexion à la base de données plus

facilement, plaçons-les dans des variables (hôte, nom de la base de données, nom d'utilisateur, mot de passe...).

```php
<?php
$dbhost = "localhost";
$dbname = "my_social_network_base";
$dbuser = "root";
$dbpassword = "root";


try
{
    $pdo = new PDO("mysql:host=$dbhost;dbname=$dbname;charset=utf8mb4",
$dbuser, $dbpassword);
    $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
}
catch (Exception $exc)
{
    die("Error while accessing database. Please contact your administrator.");
}
```

Liens de members.php vers profile.php

Arrangez-vous pour que les noms des utilisateurs dans la page `members.php` soient des liens vers leur profil. Ainsi, si l'utilisateur clique sur `ben`, il est dirigé vers la page :

http://localhost/my_social_network_base/profile.php?pseudo=ben.